

P1 · FUNÇÕES (envolvem o valor)

<code>int("123") → 123</code>	texto → inteiro; mata zero à esq. (<code>int("02") → 2</code>)
<code>float("12.5") → 12.5</code>	texto → decimal (ponto, NUNCA vírgula)
<code>str(42) → "42"</code>	número → texto
<code>input("msg")</code>	lê do usuário; retorna SEMPRE str
<code>print(x, y)</code>	mostra; vírgula põe espaço entre args
<code>len(x)</code>	tamanho: lista, string, df (nº de linhas)
<code>round(x, 2)</code>	arredonda em 2 casas
<code>sum(l) / max(l) / min(l)</code>	soma / maior / menor de lista
<code>range(5)</code>	0,1,2,3,4 (pra <code>for i in range(n)</code>)
<code>type(x)</code>	debug: qual o tipo disso?

TEU ERRO Nº1 (5x): função RETORNA, não altera. `int(x)` solto joga fora. SEMPRE `x = int(x)`. Depois de `TODO input/split`: "converso pra quê? atribuí?". Comparar/contar com `str = TypeError` ou `False` eterno (`"1" == 1` é `False`).

⚠ Preço em `input = float()`, nunca `int()` (centavos quebram `int("10.5")`).

P2 · STRINGS (.depois do valor)

<code>s.split(" ")</code>	corta → LISTA de str (converter números!)
<code>s.upper() / s.lower()</code>	MAIÚSCULA / minúscula (atribuir!)
<code>s.strip()</code>	tira espaços das pontas
<code>s.isdigit()</code>	True só se TUDO é dígito (<code>"12.3"</code> , <code>"-5"</code> , <code>"12"</code> → <code>False</code>)
<code>s.replace("a", "b")</code>	troca todas as ocorrências
<code>s[0:4]</code>	fatia: posições 0,1,2,3 (fim NÃO entra; conta do 0)
<code>"x" in s / x in lista</code>	contém?
<code>a, b, c = s.split("/")</code>	desempacota: 1 nome por pedaço (nº igual!)
<code>partes[0]</code>	caixinha 0 da lista (= coluna A do Texto p/ Colunas)

⚠ Dois separadores num `split` NÃO existe: `split` em ESTÁGIOS (espaço primeiro separa os mundos, depois / abre a data). Campo opcional: só acessar `partes[1]` DENTRO de `if len(partes)==2:`.

P3 · F-STRING (montar texto)

<code>f"Oi {nome}"</code>	f liga o modo; {} injeta variável
<code>f"{x:.2f}" / f"{x:.1f}"</code>	2 casas / 1 casa decimal

UMA string contínua. Sem `+`, sem `{}` fora do `f""`. `"Paciente" x (justaposição) = SyntaxError`. Escreve a frase literal primeiro, depois envolve as variáveis com chaves. `3.5999...` não é bug: float binário, formata com `:.2f` e segue.

P4 · CONDIÇÕES

<code>== != >= <=</code>	comparar VALOR (nunca <code>is / is not</code>)
<code>x not in ["D", "N"]</code>	"nem D nem N" (o idioma certo)
<code>x != "D" and x != "N"</code>	equivalente verboso (com <code>!=</code> o conector é AND)
<code>x >= A and x <= B</code>	"entre A e B (inclusive)": ≥ menor E ≤ maior

x != "D" or "N" é SEMPRE True ("N" sozinho é `truthy`). Cada lado do `and/or` = comparação completa. Enunciado: OU = `or`, E = `and`, literal. "Até X" INCLUI X (`<=`): borda errada = rubrica inteira (sem parcial).

P5 · LAÇOS: FOR vs WHILE

<code>for t in range(3):</code>	# nº de voltas
<code>CONHECIDO antes</code>	
<code>for item in lista:</code>	# visita cada item da coleção
<code>while saldo >= 4:</code>	# repete ENQUANTO condição valer
<code>saldo -= 4</code>	# algo muda a cada volta

⚠ Justificativa pronta: quantidade fixa antes do laço = `for`; "enquanto tiver/der" = `while`. "R\$40 dá 10 tentativas" é isca: saldo é variável, parada é condição. Bloco fecha DESINDENTANDO (`return` é de função, não fecha laço). Dentro do laço o item é `i`, nunca a lista (`split` é na fruta, não na cesta).

P6 · ESQUELETO: VALIDAÇÃO + FAIXAS

<code>d = input("Dist: ")</code>	
<code>uf = input("UF: ").upper()</code>	
<code>if not d.isdigit() or uf not in ["SP", "PR"]:</code>	
<code>print("ERRO DE ENTRADA")</code>	# valida
<code>ANTES do int()</code>	
<code>else:</code>	
<code>d = int(d)</code>	
<code>if d <= 2000:</code>	# "até X" inclui X
<code>taxa = 5.0</code>	
<code>elif d <= 8000:</code>	# elif = faixa
<code>seguinte</code>	
<code>taxa = d/1000 * 1.2</code>	# 1.2 PONTO!
<code>(1,2 = tupla)</code>	
<code>else:</code>	
<code>taxa = 25.0</code>	
<code>if uf == "PR":</code>	
<code>taxa *= 2</code>	# dobro vale p/ fixo
<code>tb</code>	
<code>print(f"R\$ {taxa:.2f}")</code>	

P7 · ESQUELETO: ACUMULADOR (Q5)

<code>cont = 0; total = 0</code>	# zera FORA
<code>for item in lista:</code>	# item = 1
<code>string</code>	
<code>partes = item.split(" ")</code>	
<code>valor = float(partes[2])</code>	# converte E atribui
<code>cont += 1</code>	
<code>total += valor</code>	
<code>print(f"... {partes[0]} ...")</code>	# 1 por item
<code>print(f"{cont} x - Total: R\$ {total:.2f}")</code>	
<code># FORA</code>	

P8 · ESQUELETO: DATA C/ CAMPO OPCIONAL

<code>meses =</code>	
<code>['janeiro', 'fevereiro', 'março', 'abril',</code>	
<code>'maio', 'junho', 'julho', 'agosto', 'setembro',</code>	
<code>'outubro', 'novembro', 'dezembro']</code>	
<code>for reg in datas:</code>	
<code>partes = reg.split(" ")</code>	#
<code>grosso 1º</code>	
<code>dia, mes, ano = partes[0].split("/")</code>	
<code>txt = f"{int(dia)} de</code>	
<code>{meses[int(mes)-1]} de {ano}"</code>	
<code>if len(partes) == 2:</code>	#
<code>guarda!</code>	
<code>h, m = partes[1].split(":")</code>	
<code>h = int(h); m = int(m)</code>	
<code>txt += " à 1 hora" if h==1 else f"</code>	
<code>às {h} horas"</code>	
<code>if m == 1: txt += " e 1 minuto"</code>	
<code>elif m > 1: txt += f" e {m}</code>	
<code>minutos"</code>	
<code>print(txt)</code>	#
<code>m==0: omite</code>	
<code># data junta: ano=s[0:4] mes=s[4:6]</code>	
<code>dia=s[6:8]</code>	

D1 · CRIAR / LER	
<code>df = pd.read_excel("a.xlsx", planilha="Excel", sheet_name="sx")</code>	
<code>pd.read_excel("a.xlsx", index_col=0)</code>	1ª coluna vira índice
<code>pd.read_csv("a.csv")</code>	CSV padrão (vírgula separa, ponto decimal)
<code>pd.read_csv("a.csv", sep=";", decimal=",")</code>	CSV BRASILEIRO: sem isso, nº vira texto
<pre>df = pd.DataFrame({ "time": ["Palmeiras", "Flamengo"], "pontoss": [68, 68]}) # dict: chave=coluna</pre>	

⚠ Teste pós-leitura: `df.dtypes` → valor tem que ser `float64/int64`; `object` = virou texto, faltou `decimals` al=",". Confira o ARQUIVO certo da questão (MrPacoca ≠ pedidos).

D2 · EXPLORAR	
<code>df.head()</code> / <code>df.tail()</code>	5 primeiras / últimas linhas (DF INTEIRO, todas col.)
<code>df.shape</code>	(linhas, colunas), sem parênteses
<code>list(df.columns)</code>	nomes EXATOS (acento conta; errado = <code>KeyError</code>)
<code>df.dtypes</code> / <code>df.info()</code>	tipos / resumo
<code>df["col"].unique()</code>	QUAIS valores existem ("Pote", não "Potes")

D3 · FILTRAR / ALTERAR / 1 LINHA	
<code>df["col"]</code>	uma coluna (Series)
<code>df["col"] == "SP"</code>	MÁSCARA: <code>True/False</code> por linha
<code>df[df["col"] == "SP"]</code>	só as linhas <code>True</code> (tabela filtrada)
<code>df.loc[mask, "col"] = v</code>	REGRA DE OURO p/ trocar valor filtrado
<code>df.loc[mask, "col"] += 0.04</code>	somar só nos filtrados
<code>df["nova"] = df["a"] * df["b"]</code>	criar coluna (vetorizado, sem laço)
<code>linha = df[df["id"]==x].iloc[0]</code>	UMA linha como ficha → linha["col"]
<code>x in df["col"].values</code>	existe? SEM <code>.values</code> checa o ÍNDICE
<code>df["col"].isin(["A", "B"])</code>	máscara: está na lista?
<code>df["col"].astype(str)</code>	converter coluna (nº ↔ texto p/ comparar)
<code>df["col"].str[0:3] / .str.upper()</code>	<code>.str</code> = métodos de string na coluna inteira (fatia SÓ em colchetes)

TABELA × LINHA × ITEM (3 crashes): "truth value of a Series is ambiguous" = coluna inteira dentro de `if`. Depois do `.iloc[0]`, TUDO vem de `linha[...]`; dentro de loop, de `i`. `df` = todo mundo · linha = um registro.

DIREÇÃO: reajuste/aumento = `+=` · desconto = `-=`. [diferença] pode coincidir e esconder o erro: confere `head()` após a troca. NUNCA `df[df[...]==x]["col"] = v` (cópia). `FutureWarning/Chained: IGNORAR` (enunciado manda).

D4 · AGREGAR / CONTAR	
<code>df["col"].sum()</code> / <code>.mean()</code> / <code>.min()</code> / <code>.max()</code>	soma, média, mín, máximo
<code>df["col"].agg(["min", "max"], round=2)</code>	vários stats em UMA operação (exigência típica)
<code>df["col"].value_counts()</code>	contagem POR categoria (NÃO fazer loop c/ contadores!)
<code>df["col"].nunique()</code>	QUANTAS categorias distintas
<code>mask.sum()</code>	conta <code>True</code> (<code>True=1</code>). <code>.count()</code> = não-nulos, NÃO use
<code>serie.sort_values(ascending=False)</code>	decrecente; <code>.head(5)</code> = top 5
<code>df["col"].describe()</code>	pacote completo: count, mean, std, min, quartis, max

D5 · ANÁLISE (Excel → pandas)	
<code>df.groupby("g")["v"].mean().sort_values(ascending=False)</code>	Tabela Dinâmica de 1 medida = ranking por grupo
<code>df.groupby("g")["v"].agg(["mean", "max"])</code>	vários stats por grupo
<code>pd.merge(a, b, on=["k1", "k2"])</code>	PROCV por N chaves; chave faltando MULTIPLICA linhas (teste: <code>shape</code>)
<code>df.pivot_table(index="Estado", columns="Produto", values="Total", aggfunc="mean").reset_index()</code>	Tabela Dinâmica: index=Linhas, columns=Colunas, values=Valores, <code>aggfunc=Resumir</code> por <code>df.columns.name=None</code> limpa rótulo
<code>(df["m"] < 0).replace({True: "prejuízo", False: "lucro"})</code>	classificar sem <code>if</code>
<code>np.where(cond, "sim", "não")</code>	idem (import <code>numpy as np</code>)

⚠ TRIAGEM: dataset grande + média/ranking/cruzar = pandas SEM `for`. Lista de strings item a item = `for` + `split` (lado PYTHON). Ranking liderado por grupo de amostra MÍNIMA = liderança frágil: olha o `value_counts()` antes de confiar.

D7 · ESQUELETO: REAJUSTE + DIFERENÇA	
<code>total_pre = df["Total"].sum()</code>	# guarda ANTES
<code>df["Preço Unitário"] = df["Total"]/df["Quantidade"]</code>	
<code>df.loc[df["Mês"]==mes, "Preço Unitário"] += reajuste</code>	
<code>df["Total"] = df["Preço Unitário"]*df["Quantidade"]</code>	
<code>print(f'R\$ {df["Total"].sum() - total_pre:.2f}')</code>	
# usa as VARIÁVEIS dadas, nunca os nº do exemplo	

D6 · ESQUELETO: BUSCA ID + ÁRVORE	
<code>pid = int(input("id: "))</code>	# int: coluna é nº!
<code>if pid not in df["id"].values:</code>	<code>print("Paciente não encontrado")</code>
<code>else:</code>	
<code> linha = df[df["id"]==pid].iloc[0]</code>	
<code> imc = linha["peso"]/linha["altura"]**2</code>	
<code> if linha["sist"] >= 140 or linha["diast"] >= 90:</code>	
<code> r = "consulta imediata"</code>	# regra 1
<code>GANHA</code>	
<code> elif imc >= 30 and linha["fum"]=="sim":</code>	<code>r = "risco alto"</code> # ordem
<code>elif imc < 30:</code>	<code>r = "risco moderado"</code> # OU do
<code>enunciado = or</code>	
<code> else:</code>	
<code> r = "risco baixo"</code>	
<code> print(f"Paciente {pid} (IMC: {imc:.1f}) - Risco: {r}")</code>	
# travessão - : COPIA do enunciado	

🔥 PROTOCOLO (10 passos)	
0. Q0 primeiro	nome, turma AE_, código; não apagar os #
1. Triagem 60s	cada questão: conceito / ler pandas / if+validação / DataFrame / parse em laço / análise
2. Ordem	Q1 → Q2 → Q5 (3pts) → Q3 → Q4. Trouvou 10min: pula e volta
3. Antes do código	<code>df.columns + head()</code> : nomes e valores REAIS
4. Consertos em LOTE	achou 3 problemas? conserta os 3, DEPOIS roda
5. Teste de fechamento	exemplo do enunciado + borda exata de cada faixa + caso de ERRO + UMA entrada de CADA categoria/estado
6. Warnings	<code>SyntaxWarning</code> DIZ o conserto; <code>FutureWarning</code> : ignora
7. Saída completa	pediu 3 valores? imprime os 3 (faltar 1 = rubrica zero)
8. Formato exato	copia símbolo do enunciado · <code>.2f</code> · linha em branco do modelo
9. Run All	Restart & Run All sem erro (prosa em célula de código = TODA linha com #)
10. Varredura final	NENHUMA resposta em branco · Q0 preenchida